PARTE I REQUISITOS COMUNS AOS 13 GRUPOS

Apresentação

Este documento estabelece os requisitos técnicos, funcionais, visuais e documentais que deverão ser atendidos por todos os 13 grupos. Cada grupo desenvolverá um domínio diferente, mas todos utilizarão o projeto `mural-servidor-aula-02-completo` como referência de organização e deverão demonstrar os mesmos conhecimentos fundamentais.

Os wireframes específicos determinam a composição visual de cada projeto. Este documento não substitui esses wireframes: ele define as obrigações comuns que garantem equivalência de complexidade, critérios comparáveis e condições justas de avaliação.

Resultado esperado

- uma aplicação web executada em servidor Node.js com Express;
- dados persistidos em SQLite e manipulados com Sequelize;
- páginas renderizadas no servidor com Handlebars;
- página inicial, listagem, detalhamento e erro 404 coerentes com o wireframe do grupo;
- tratamento de falhas internas, arquivos estáticos, testes automatizados e documentação;
- código adaptado ao domínio do grupo, sem referências residuais indevidas ao mural de publicações.

1 Objetivo da avaliação

A avaliação verificará se o grupo consegue compreender, adaptar e explicar uma aplicação web completa. A tarefa não consiste em apenas substituir textos, nomes e cores. O grupo deverá modificar a entidade, o banco de dados, as rotas, as consultas, as views, os partials, os testes e a organização visual de acordo com o projeto recebido.

1.1 Conhecimentos avaliados

- criação e configuração de um projeto Node.js e gerenciamento de dependências com NPM;
- configuração do Express e separação entre aplicação e inicialização do servidor;
- definição de rotas e utilização de parâmetros de rota;
- criação e utilização de middlewares;
- renderização com Handlebars, layouts, views, partials, condicionais e laços;
- modelagem de dados com Sequelize e persistência com SQLite;
- consultas com `count()`, `bulkCreate()`, `findAll()` e `findByPk()`;
- disponibilização de CSS, imagens e outros arquivos estáticos;
- tratamento de erros 404 e 500;
- organização inspirada no padrão MVC;
- criação de testes automatizados com Node Test e Supertest.

2 Estrutura mínima do projeto

A nomenclatura deverá representar a entidade do grupo. O termo `entidade` é apenas um marcador e deverá ser substituído por aviso, livro, evento, oportunidade ou outro nome correspondente.

```
nome-do-projeto/
├── app.js
├── server.js
├── package.json
├── package-lock.json
├── README.md
├── models/
│   ├── index.js
│   └── entidade.js
├── routes/
│   ├── index.js
│   └── entidades.js
├── views/
│   ├── layouts/main.hbs
│   ├── partials/componente-entidade.hbs
│   ├── entidades/listar.hbs
│   ├── entidades/detalhar.hbs
│   ├── index.hbs
│   ├── 404.hbs
│   └── erro.hbs
├── public/
│   ├── images/
│   └── stylesheets/style.css
└── test/app.test.js
```

Não deverão permanecer arquivos ou identificadores como `publicacao.js`, `publicacoes.js` e `cartao-publicacao.hbs` quando eles não representarem o domínio atribuído ao grupo.

3 Responsabilidade das partes

Parte	Responsabilidade
server.js	Sincronizar o banco, preparar dados iniciais e iniciar o servidor.
app.js	Configurar Express, Handlebars, arquivos estáticos, middlewares, rotas e erros.
models	Definir a entidade, os campos, tipos, validações e acesso aos dados.
routes	Receber requisições, validar parâmetros, consultar Models e selecionar a view.
views	Produzir as páginas apresentadas ao usuário.
partials	Reutilizar componentes visuais repetidos nas páginas.
public	Armazenar CSS, imagens e recursos acessíveis ao navegador.
test	Verificar automaticamente respostas, conteúdo e situações de erro.

A organização é inspirada no padrão MVC. O Model concentra dados; as rotas exercem a função de controle da requisição; e as views apresentam a resposta. O grupo deverá evitar consultas ao banco diretamente nos templates e evitar HTML dentro do Model.

4 Dependências e comandos

O projeto deverá possuir as dependências necessárias registradas no `package.json` e no `package-lock.json`.

```
npm install express express-handlebars sequelize sqlite3
npm install --save-dev supertest
```

4.1 Scripts obrigatórios

```
"scripts": {
  "start": "node server.js",
  "dev": "node --watch server.js",
```

```
  "test": "node --test"
}
```

- `npm start` deverá iniciar a aplicação sem exigir alteração manual de código;
- `npm test` deverá executar a suíte de testes;
- a porta poderá vir de `process.env.PORT` e possuir um valor padrão;
- a inicialização deverá informar o endereço local da aplicação.

5 Páginas e navegação obrigatórias

Página	Finalidade	Obrigação principal
Inicial	Contextualizar o sistema	Apresentar título, explicação, ação principal e três destaques vindos do banco.
Listagem	Exibir os registros	Utilizar a organização prevista no wireframe e gerar os componentes dinamicamente.
Detalhamento	Exibir um registro	Receber :id, consultar o banco e apresentar todos os campos obrigatórios.
Erro 404	Tratar conteúdo inexistente	Contextualizar a mensagem e oferecer retorno seguro.
Erro 500	Tratar falha interna	Ocultar detalhes técnicos e informar que ocorreu uma falha inesperada.

5.1 Navegação

- o cabeçalho deverá apresentar a identidade do projeto;
- os links do menu deverão funcionar em todas as páginas;
- a página inicial deverá conduzir à listagem;
- cada componente da listagem deverá conduzir ao detalhamento correspondente;
- o detalhamento e a página 404 deverão possuir ação de retorno;
- não poderão existir links com `href` vazio, # sem função ou destinos inexistentes.

6 Rotas obrigatórias

O substantivo `entidades` deverá ser substituído pelo recurso do grupo.

Método e rota	Resposta esperada
GET /	Renderizar a página inicial.
GET /saude	Responder status 200 e JSON { "status": "ok" }.
GET /entidades	Consultar e renderizar todos os registros.
GET /entidades/:id	Consultar e renderizar um registro específico.
rota desconhecida	Responder status 404 e renderizar a página contextualizada.

6.1 Validação do parâmetro id

1. Ler `req.params.id`.
2. Converter o valor recebido para número.
3. Verificar se o valor é um número inteiro positivo.
4. Consultar o banco com `findByPk()`.
5. Responder 404 quando o identificador for inválido ou o registro não existir.
6. Encaminhar exceções inesperadas ao middleware de erro com `next(erro)`.

```
const id = Number(req.params.id);

if (!Number.isInteger(id) || id <= 0) {
  return res.status(404).render("404");
}
```

```
const registro = await Entidade.findByPk(id);

if (!registro) {
  return res.status(404).render("404");
}
```

7 Modelagem e banco de dados

Cada projeto deverá possuir uma entidade principal coerente com o domínio. Os campos particulares estão definidos na instrução específica do grupo. Todos os campos deverão utilizar tipos compatíveis com o conteúdo armazenado.

- `STRING` para títulos, categorias, nomes e informações curtas;
- `TEXT` para descrições, mensagens, resumos e sinopses extensas;
- `INTEGER` para quantidades, capacidade, carga horária, vagas e ano;
- `DATE` ou `DATEONLY` para datas;
- `BOOLEAN` somente quando o domínio possuir uma decisão verdadeira ou falsa;
- `allowNull: false` para informações realmente obrigatórias.

7.1 Dados iniciais

O projeto deverá possuir pelo menos seis registros iniciais diferentes. Antes da inserção, a aplicação deverá verificar se a tabela está vazia, evitando duplicação a cada reinicialização.

```
const quantidade = await Entidade.count();

if (quantidade === 0) {
  await Entidade.bulkCreate([
    // seis ou mais registros coerentes
  ]);
}
```

- os dados deverão representar todos os campos obrigatórios;
- os registros deverão permitir observar ordenação e indicadores visuais;
- não utilizar “teste”, “exemplo”, “lorem ipsum” ou “qualquer coisa” como conteúdo final;
- não criar seis cópias do mesmo registro;
- os dados não poderão ser substituídos por um array fixo nas rotas ou views.

7.2 Consultas

```
const registros = await Entidade.findAll({
  order: [["campo", "ASC"]]
});

const registro = await Entidade.findByPk(id);
```

A ordenação deverá seguir a instrução específica do projeto. Quando o domínio não definir uma ordem natural, o grupo deverá justificar sua escolha no README.

8 Handlebars e renderização

Recurso	Uso obrigatório
Layout	Estrutura comum com HTML, cabeçalho, navegação, main e rodapé.
View inicial	Apresentação do domínio e registros em destaque.

Recurso	Uso obrigatório
View de listagem	Organização prevista no wireframe e repetição dos registros.
View de detalhamento	Campos do registro selecionado e ação de retorno.
Partial	Unidade visual repetida, como cartão, linha, perfil ou item cronológico.
{{#each}}	Percorrer destaques e registros.
{{#if}}	Tratar estados, indicadores ou ausência de registros.
Helper	Formatar pelo menos um dado, preferencialmente data, horário ou texto.

8.1 Destaques da página inicial

Os três destaques visíveis nos wireframes deverão vir do banco.

```
const destaques = await Entidade.findAll({
  limit: 3
});

res.render("index", { destaques });
```

8.2 Ausência de registros

```
{{#if registros.length}}
  {{#each registros}}
    {{> componente-entidade}}
  {{/each}}
{{else}}
  <p>Nenhum registro foi encontrado.</p>
{{/if}}
```

9 Middlewares

A aplicação deverá utilizar middlewares do Express e pelo menos um middleware próprio. A ordem de registro deverá respeitar o fluxo da requisição.

- `express.json()` para interpretar JSON;
- `express.urlencoded()` para interpretar dados de formulários;
- `express.static()` para disponibilizar `public`, CSS e imagens;
- middleware próprio para registrar método, endereço e, opcionalmente, horário;
- middleware 404 depois de todas as rotas válidas;
- middleware de erro 500 por último e com quatro parâmetros.

```
app.use((req, res, next) => {
  console.log(`[${req.method}] ${req.originalUrl}`);
  next();
});
```

O middleware próprio deverá chamar `next()`. Sem essa chamada, a requisição poderá permanecer interrompida e o navegador não receberá resposta.

10 Tratamento de erros

Situação	Status	Comportamento
Rota inexistente	404	Renderizar 404.hbs com mensagem contextualizada.
Identificador inválido	404	Evitar consulta inadequada e renderizar 404.
Registro não encontrado	404	Informar que a entidade específica não foi localizada.
Exceção inesperada	500	Registrar detalhes no servidor e renderizar erro.hbs.

A interface não deverá exibir stack trace, instruções SQL, caminhos internos ou detalhes de configuração. Informações técnicas deverão permanecer no terminal do servidor.

11 Fidelidade aos wireframes

Os wireframes são obrigatórios como referência de arquitetura da informação. Não se exige reprodução pixel a pixel, mas a solução deverá preservar a composição e a hierarquia específicas do projeto.

11.1 Elementos que deverão ser preservados

- formato geral da página inicial: dividida, centralizada ou em painel;
- tipo de listagem: cartões, linhas, catálogo, fórum, linha do tempo ou colunas;
- posição e relevância dos indicadores;
- organização geral do detalhamento;
- presença de imagem ou área visual quando prevista;
- mensagem contextualizada e composição geral da página 404;
- percurso entre página inicial, listagem, detalhamento e retorno.

11.2 Elementos que poderão ser adaptados

- tonalidades dentro da identidade temática;
- fontes legíveis e adequadas;
- ícones e imagens;
- espaçamentos e dimensões para responsividade;
- textos complementares;
- pequenos ajustes necessários para melhorar acessibilidade.

11.3 Marcadores que não são conteúdo final

As expressões “IMAGEM”, “ÁREA VISUAL”, “Valor demonstrativo”, “ICONE” e “Registro ativo” presentes nos wireframes são instruções estruturais. Elas deverão ser substituídas por imagens, ícones, valores e rótulos reais.

12 Responsividade e acessibilidade

12.1 Responsividade

- a aplicação deverá permanecer utilizável em computador, tablet e celular;
- grades deverão reduzir a quantidade de colunas conforme a largura disponível;
- painéis laterais poderão ser posicionados abaixo do conteúdo principal;
- as duas colunas de uma agenda poderão ser empilhadas;
- textos, imagens e botões não poderão ultrapassar seus componentes;
- o menu deverá continuar acessível, mesmo sem menu hambúrguer.

12.2 Acessibilidade mínima

- atributo `lang="pt-BR"` no documento HTML;
- uso coerente de `header`, `nav`, `main` e `footer`;
- hierarquia de títulos sem saltos injustificados;
- contraste suficiente entre texto e fundo;
- texto alternativo para imagens informativas;
- foco visível em links e botões;

- navegação possível por teclado;
- textos de links que indiquem claramente o destino;
- informações de estado que não dependam exclusivamente de cor.

13 Arquivos estáticos e identidade visual

- todo CSS deverá ser carregado a partir de `public/stylesheets`;
- imagens locais deverão ficar em `public/images`;
- não utilizar caminhos absolutos do computador do estudante;
- a identidade visual deverá corresponder ao tema atribuído;
- elementos reutilizados deverão compartilhar classes CSS;
- não utilizar estilos inline como solução principal;
- a interface deverá manter legibilidade, alinhamento e espaçamento consistentes.

14 Testes automatizados

Cada grupo deverá adaptar os testes ao próprio domínio. Não poderão permanecer verificações de textos, rotas ou entidades do mural original.

Teste	Resultado esperado
GET /	Status 200 e título do projeto.
GET /saude	Status 200 e JSON com status: ok.
GET /entidades	Status 200 e conteúdo de um registro inicial.
GET /entidades/:id existente	Status 200 e conteúdo do registro solicitado.
GET /entidades/:id inexistente	Status 404.
GET /entidades/valor-invalido	Status 404.
GET /rota-inexistente	Status 404 e página contextualizada.

```
test("deve apresentar a listagem", async () => {
  const resposta = await request(app)
    .get("/entidades");

  assert.equal(resposta.status, 200);
  assert.match(resposta.text, /Registro esperado/);
});
```

15 Documentação obrigatória

O `README.md` deverá permitir que uma pessoa que não participou do desenvolvimento instale, execute e compreenda a aplicação.

- nome do projeto e integrantes;
- problema atendido e público beneficiado;
- entidade principal e campos;
- tecnologias utilizadas;
- pré-requisitos;
- instalação das dependências;
- execução do servidor;
- execução dos testes;
- rotas disponíveis;

- organização dos diretórios;
- correspondência entre a aplicação e os quatro wireframes;
- decisões e adaptações realizadas pelo grupo.

16 Organização do trabalho do grupo

Responsabilidade inicial	Atividades
Model e banco	Definir entidade, conexão, sincronização, campos e dados iniciais.
Rotas e controle	Implementar consultas, validações, parâmetros e erros.
Views e Handlebars	Construir layout, views, partials, each, if e helper.
Interface e qualidade	Aplicar CSS, responsividade, acessibilidade, testes e documentação.

A divisão organiza o início do trabalho, mas não isenta nenhum integrante de compreender o projeto completo. Todos deverão participar da integração, dos testes e da revisão final.

17 Restrições

Não serão aceitas como atendimento integral da avaliação:

- aplicação composta somente por HTML e CSS estáticos;
- dados escritos manualmente nos cartões das views;
- substituição do SQLite por um array fixo;
- criação manual de um componente para cada registro;
- remoção do detalhamento por identificador;
- remoção do tratamento de erros;
- abandono de Handlebars ou Sequelize;
- cópia do projeto-base sem adaptação técnica;
- interface genérica incompatível com o wireframe do grupo;
- uso de código que nenhum integrante consiga explicar.

18 Entrega

O grupo deverá entregar uma pasta organizada e testada. A entrega deverá conter:

- código-fonte completo;
- `package.json` e `package-lock.json`;
- Models, rotas, views, partials, CSS, imagens e testes;
- mecanismo de criação ou preparação do banco;
- seis ou mais registros iniciais;
- README completo;
- capturas da página inicial, listagem, detalhamento e erro 404;
- arquivo compactado nomeado conforme orientação do professor.

19 Definição de projeto concluído

O projeto somente será considerado concluído quando:

- `npm install` for concluído sem erro;
- `npm start` iniciar o servidor;
- o banco for sincronizado e os dados iniciais não forem duplicados;
- a página inicial apresentar dados do banco;
- a listagem apresentar todos os registros;
- cada ação de detalhes abrir o registro correto;
- identificadores inválidos e inexistentes produzirem 404;
- falhas internas forem tratadas de forma segura;
- CSS e imagens forem carregados;
- a interface corresponder aos wireframes do grupo;
- a aplicação funcionar em telas menores;
- `npm test` for executado sem falhas;
- não existirem referências residuais indevidas ao projeto-base;
- todos os integrantes conseguirem explicar o fluxo da aplicação.

20 Checklist final do grupo

- ☐ A entidade e seus campos correspondem ao projeto atribuído.
- ☐ Os arquivos e variáveis foram renomeados de forma coerente.
- ☐ A página inicial consulta e apresenta destaques do banco.
- ☐ A listagem utiliza `findAll()` e possui ordenação.
- ☐ O detalhamento utiliza `findByPk()` e `req.params.id`.
- ☐ O parâmetro `id` é validado antes da consulta.
- ☐ Existem seis ou mais registros iniciais diferentes.
- ☐ Os dados não são duplicados quando o servidor reinicia.
- ☐ A listagem utiliza `{{#each}}` e um partial.
- ☐ Existe pelo menos uma condicional `{{#if}}` útil.
- ☐ Existe pelo menos um helper.
- ☐ O formato da listagem corresponde ao wireframe específico.
- ☐ O detalhamento preserva a hierarquia prevista.
- ☐ A página 404 está contextualizada.
- ☐ Existe tratamento de erro 500.
- ☐ Os middlewares estão registrados na ordem correta.
- ☐ Links, botões e retornos funcionam.
- ☐ A interface é responsiva e acessível por teclado.
- ☐ Imagens possuem texto alternativo.
- ☐ Os testes utilizam rotas e conteúdos do domínio correto.
- ☐ `npm test` é executado sem falhas.
- ☐ O README explica instalação, execução, rotas e decisões.
- ☐ O arquivo final foi aberto e testado antes do envio.

Síntese do fluxo técnico

O grupo deverá ser capaz de explicar o percurso completo de uma requisição: o navegador solicita um endereço; o Express recebe a requisição; os middlewares realizam tarefas intermediárias; a rota valida os dados recebidos; o

Model utiliza o Sequelize para consultar o SQLite; a rota envia os dados para a view; o Handlebars produz o HTML; e o Express devolve a resposta ao navegador.

A qualidade do trabalho será observada tanto no funcionamento quanto na capacidade do grupo de justificar as decisões adotadas e relacioná-las aos conhecimentos estudados na Aula 02.

PARTE II INSTRUÇÕES EXCLUSIVAS DO GRUPO 05

Oportunidades

1 Missão do grupo

O grupo deverá desenvolver uma aplicação denominada **Oportunidades**. A entidade principal será **Oportunidade**. O sistema deverá apresentar oportunidades, permitir a consulta da listagem, abrir o detalhamento de um registro por identificador e tratar endereços ou registros inexistentes.

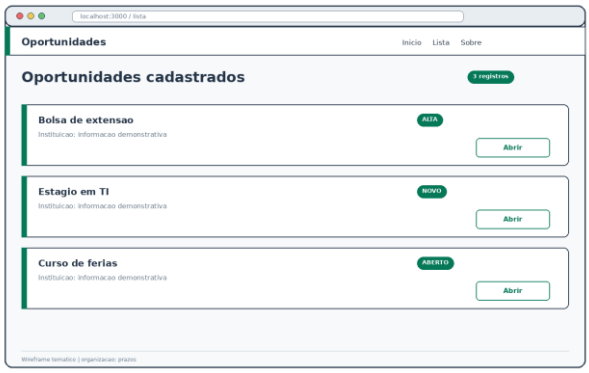
O objetivo do projeto é divulgar bolsas, estágios, cursos e processos seletivos relevantes para estudantes. A implementação deverá respeitar o projeto-base e, ao mesmo tempo, possuir identidade, dados, rotas, componentes e testes próprios.

2 Referência visual obrigatória

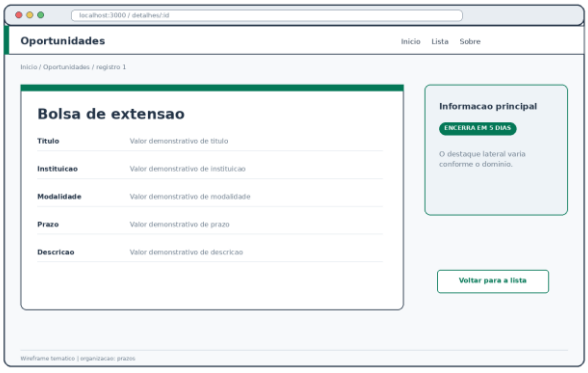
Os quatro wireframes a seguir correspondem exclusivamente a este grupo. Eles definem a hierarquia, a composição e o percurso de navegação. As palavras IMAGEM, ÁREA VISUAL, ICONE, Registro ativo e Valor demonstrativo são marcadores e deverão ser substituídas por conteúdo real.



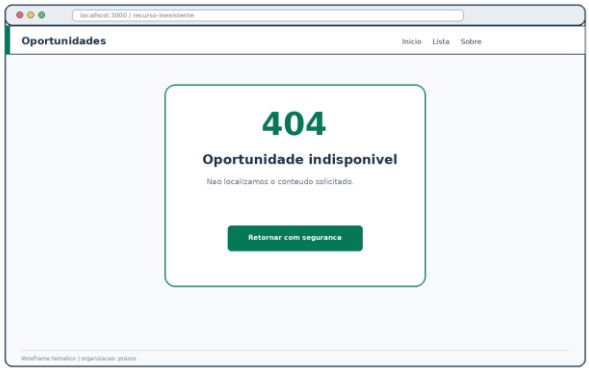
Página inicial



Listagem



Detalhamento



Erro 404

3 Entidade e campos obrigatórios

A entidade Sequelize deverá ser denominada `Oportunidade` e armazenada no arquivo `models/oportunidade.js`. Essa nomenclatura deverá permanecer consistente nas importações, nas rotas e nos testes.

Campo	Tipo sugerido	Finalidade
titulo	STRING	Identificar a oportunidade.
instituicao	STRING	Informar quem oferece.
modalidade	STRING	Classificar bolsa, estágio, curso ou seleção.
prazo	DATE ou DATEONLY	Registrar a data limite.
descricao	TEXT	Explicar requisitos e finalidade.

Todos os campos apresentados acima deverão aparecer na aplicação. Campos adicionais somente deverão ser criados quando possuírem finalidade clara e forem explicados no README.

4 Rotas exclusivas

- `GET /`: página inicial do projeto;
- `GET /saude`: diagnóstico em JSON;
- `GET /oportunidades`: listagem de oportunidades;
- `GET /oportunidades/:id`: detalhamento de um registro;
- qualquer rota desconhecida: página 404 contextualizada.

O arquivo de rotas deverá possuir nome coerente, preferencialmente `routes/oportunidades.js`. A aplicação não deverá continuar utilizando `/publicacoes` como rota principal.

5 Dados iniciais

O banco deverá possuir pelo menos seis registros. Utilize, no mínimo, os seguintes conteúdos como referência:

- Bolsa de extensão;
- Estágio em TI;
- Curso de férias;
- Programa de monitoria;
- Seleção para projeto;
- Oficina externa.

Os registros deverão preencher todos os campos, apresentar valores diferentes e produzir uma listagem visualmente significativa. A ordenação obrigatória será **prazo em ordem crescente**.

6 Página inicial

A página inicial deverá seguir a organização de **composição centralizada**.

Centralizar título, apresentação e botão. Abaixo do separador, mostrar três oportunidades em destaque.

Os três destaques deverão vir do banco de dados. O botão principal deverá direcionar para `/oportunidades`. A identidade poderá utilizar verde como referência, mantendo contraste e legibilidade.

Verificação da página inicial

- apresenta o nome Oportunidades;
- explica a finalidade do sistema;
- contém ação principal funcional;
- apresenta três registros recuperados do banco;
- corresponde ao primeiro wireframe;
- permanece utilizável em telas menores.

7 Página de listagem

A listagem deverá utilizar **linhas horizontais com destaque para prazo**.

Cada linha deverá apresentar título, instituição, modalidade, prazo, selo de situação e ação. O prazo precisa ser uma das informações mais visíveis.

A rota deverá consultar todos os registros com `findAll()`, aplicar prazo em ordem crescente e enviar a coleção para a view. A view deverá utilizar `{{#each}}` e um partial. O indicador visual principal será **situação do prazo**.

O botão ou link de cada componente deverá apontar para:

```
href="/oportunidades/{{id}}"
```

Quando a coleção estiver vazia, apresentar uma mensagem relacionada ao domínio, como “Nenhum registro foi encontrado”.

8 Página de detalhamento

O detalhamento deverá seguir a organização de **painel principal com prazo destacado lateralmente**.

Apresentar todos os campos no painel principal. A lateral deve destacar a data limite ou uma mensagem como Encerra em 5 dias.

A rota deverá validar `req.params.id`, utilizar `findByPk()` e responder com status 404 quando o identificador for inválido ou inexistente. A área de maior destaque lateral deverá apresentar **prazo ou quantidade de dias restantes**.

Todos os campos obrigatórios deverão ser exibidos com rótulos compreensíveis. O botão de retorno deverá apontar para `/oportunidades`.

9 Página de erro 404

O wireframe determina **cartão central contornado**. A mensagem principal deverá ser:

```
404
Oportunidade indisponível
```

A página deverá explicar que o endereço ou identificador informado não corresponde a um registro disponível e oferecer retorno para `/oportunidades`. Não apresentar stack trace, consulta SQL ou detalhes internos.

10 Partial obrigatório

O grupo deverá criar um partial correspondente ao componente repetido na listagem. Sugestão de nome:

```
views/partials/componente-oportunidades.hbs
```

O nome poderá ser simplificado, desde que represente o domínio. O partial deverá receber um registro por vez e não poderá conter dados fixos específicos de apenas um item.

11 Helper e condicional

O projeto deverá possuir pelo menos um helper útil. Para projetos com datas, recomenda-se `formatarData`. Para horários, textos ou indicadores, o grupo poderá criar outro helper coerente.

Também deverá existir pelo menos uma condicional `{{#if}}` para tratar lista vazia, indicador, situação ou conteúdo opcional.

12 Imagens e elementos visuais

A identidade temática deverá utilizar verde como referência. Isso não impede outras cores de apoio. Os indicadores não poderão depender somente da cor: deverão possuir texto visível.

Quando o wireframe apresentar imagem, capa ou fotografia, utilizar arquivos locais em `public/images` e caminhos públicos iniciados por `/images/`. Quando apresentar ícone, poderá ser utilizado símbolo comum ao componente, desde que seja acessível e coerente.

13 Testes específicos

Além dos testes comuns, o grupo deverá comprovar:

- que `GET /oportunidades` responde com status 200;
- que a listagem contém pelo menos um dos registros iniciais;
- que `GET /oportunidades/1` abre um registro existente;
- que `GET /oportunidades/99999` responde com status 404;
- que `GET /oportunidades/abc` responde com status 404;
- que a página 404 contém a mensagem “Oportunidade indisponível”;
- que os testes não procuram textos ou rotas do mural de publicações.

14 Sequência recomendada de implementação

1. Copiar e executar o projeto-base sem modificações.
2. Renomear o Model e definir os campos de `Oportunidade`.
3. Substituir os dados iniciais por registros de oportunidades.
4. Criar `routes/oportunidades.js` e registrar a rota no `app.js`.
5. Implementar a consulta de destaques da página inicial.
6. Implementar a listagem com `findAll()` e ordenação.
7. Criar o partial correspondente ao formato do wireframe.
8. Implementar o detalhamento com `findByPk()`.
9. Validar identificadores inválidos e inexistentes.
10. Adaptar `index.hbs`, a listagem, o detalhamento e o erro 404.
11. Aplicar CSS e responsividade conforme os quatro wireframes.
12. Criar helper, condicional e mensagens de ausência de dados.
13. Adaptar e ampliar os testes.
14. Revisar acessibilidade, README, nomes de arquivos e referências residuais.
15. Executar `npm test`, reiniciar o servidor e conferir que os dados não duplicam.

15 Checklist exclusivo do Grupo 05

- ☐ A aplicação utiliza a entidade `Oportunidade`.
- ☐ A rota principal da coleção é `/oportunidades`.
- ☐ Os campos obrigatórios foram implementados.
- ☐ Existem pelo menos seis registros coerentes.
- ☐ A página inicial corresponde ao wireframe do grupo.
- ☐ Os três destaques vêm do banco.
- ☐ A listagem utiliza linhas horizontais com destaque para prazo.
- ☐ A ordenação utiliza prazo em ordem crescente.
- ☐ O indicador de situação do prazo está visível e textual.
- ☐ O detalhamento utiliza painel principal com prazo destacado lateralmente.

- ☐ A área lateral destaca prazo ou quantidade de dias restantes.
- ☐ A página 404 apresenta “Oportunidade indisponível”.
- ☐ O botão da página 404 retorna para `/oportunidades`.
- ☐ O partial foi adaptado ao domínio.
- ☐ O CSS utiliza identidade coerente com verde.
- ☐ Os testes utilizam as rotas e os textos deste projeto.
- ☐ Não permanecem referências indevidas a publicações.

16 Critério de conclusão do projeto

O projeto será considerado concluído quando as quatro telas estiverem implementadas, os dados forem recuperados do SQLite por meio do Sequelize, a navegação funcionar, os erros forem tratados, os testes forem aprovados e todos os integrantes conseguirem explicar como a requisição percorre middleware, rota, Model e view até produzir a resposta HTML.